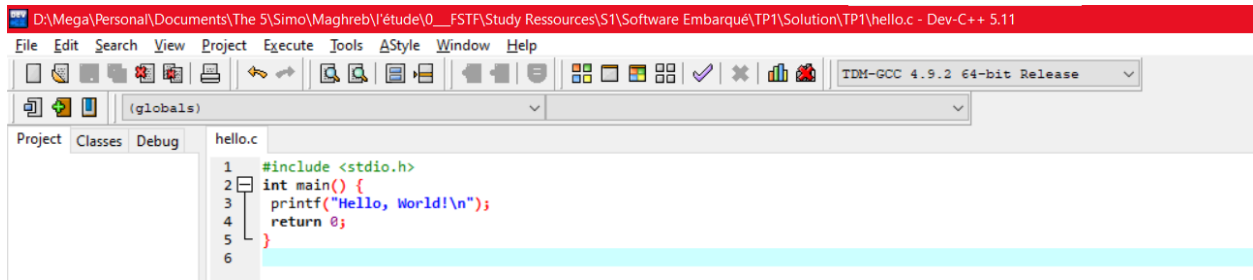


Cette expérience nous a permis de comprendre les principes fondamentaux de la programmation en C, qui sont essentiels dans le développement embarqué et dans la gestion de projets comportant plusieurs fichiers sources.

Exercice 1

Dans cet exercice, on écrit un programme C simple qui affiche le message **"Hello, World!"** à l'écran.

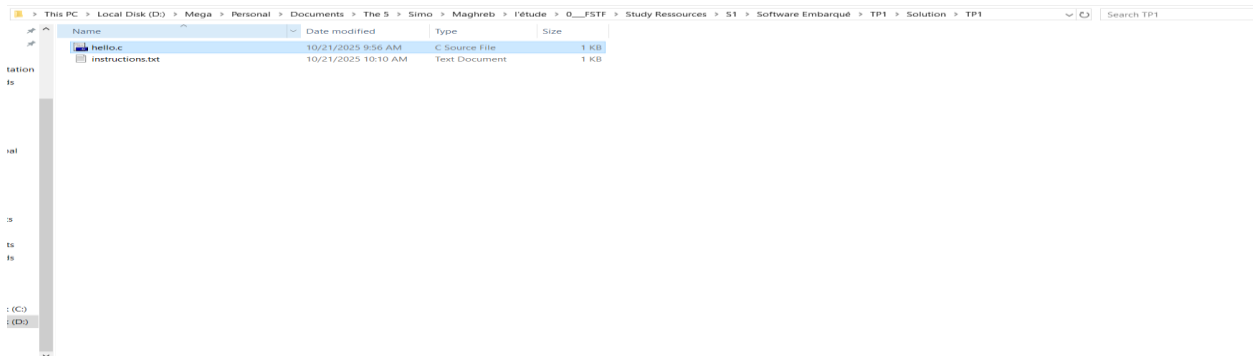
Le fichier source **hello.c** est compilé avec la commande :



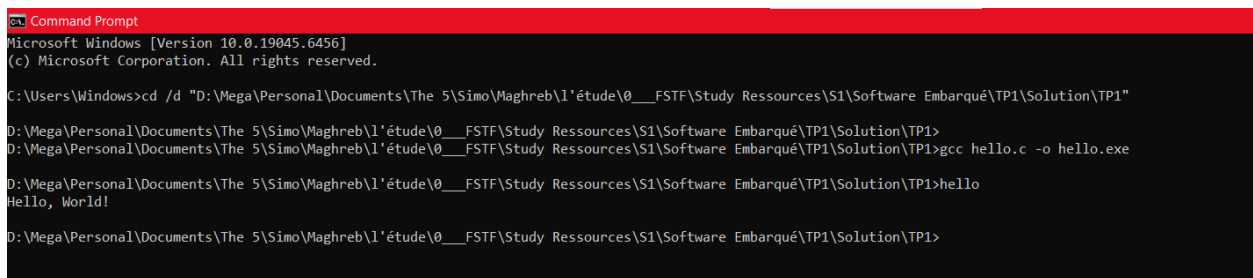
The screenshot shows the Dev-C++ 5.11 IDE. The title bar indicates the file path: D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1\hello.c - Dev-C++ 5.11. The menu bar includes File, Edit, Search, View, Project, Execute, Tools, AStyle, Window, and Help. The toolbar contains various icons for file operations and execution. The compiler is set to TDM-GCC 4.9.2 64-bit Release. The Project, Classes, and Debug tabs are visible. The main editor window displays the source code of hello.c:

```
1 #include <stdio.h>
2 int main() {
3     printf("Hello, World!\n");
4     return 0;
5 }
6
```

On compile le code avec la commande : « **gcc hello.c -o hello** »



Le résultat affiché est :



The screenshot shows a Windows Command Prompt window. The title bar is 'Command Prompt'. The text in the window is as follows:

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows>cd /d "D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1"

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1>
D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1>gcc hello.c -o hello.exe

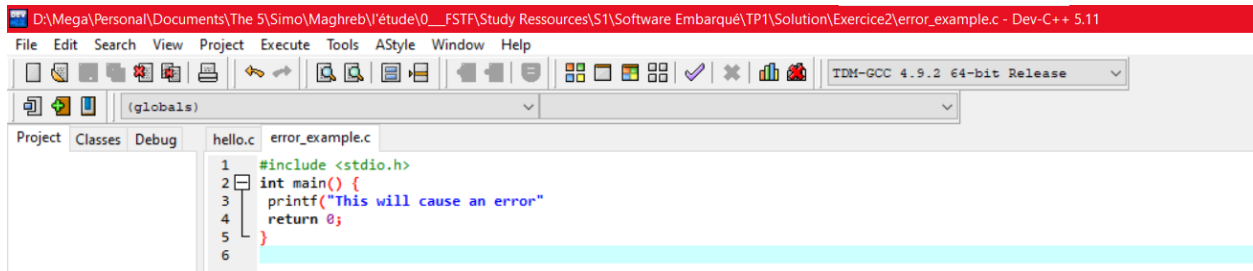
D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1>hello
Hello, World!

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\TP1>
```

Exercice 2

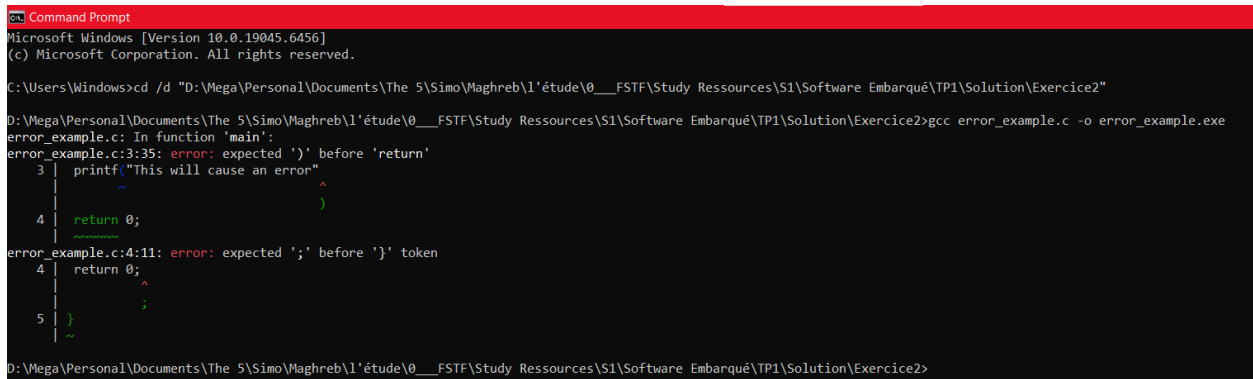
Un programme contenant une erreur de syntaxe a été compilé afin d'observer le message d'erreur produit par le compilateur GCC.

Le fichier en question s'appelle **error_example.c**.



```
D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2\error_example.c - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug hello.c error_example.c
1 #include <stdio.h>
2 int main() {
3     printf("This will cause an error"
4     return 0;
5 }
6
```

Après l'exécution de la commande : **gcc error_example.c -o error_example**, nous avons obtenu l'erreur suivante générée par GCC :

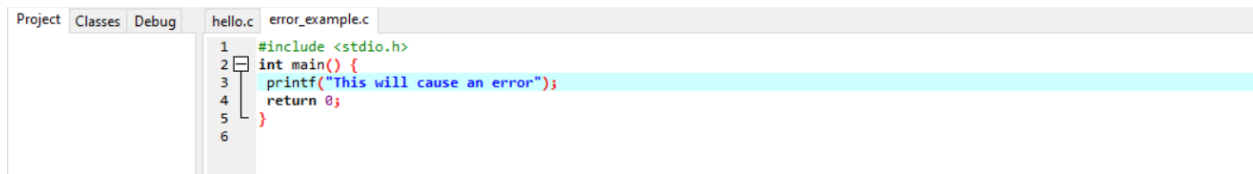


```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows>cd /d "D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2"

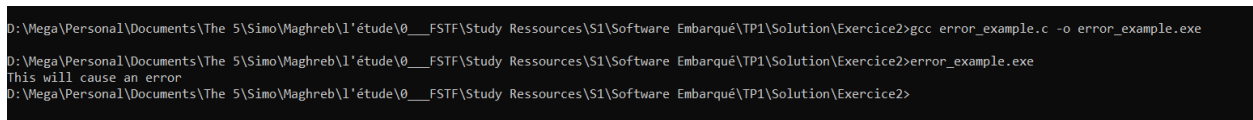
D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2>gcc error_example.c -o error_example.exe
error_example.c: In function 'main':
error_example.c:3:35: error: expected ')' before 'return'
   3 |     printf("This will cause an error"
     |                                   ^
   4 |     return 0;
     |     ~~~~~
error_example.c:4:11: error: expected ';' before '}' token
   4 |     return 0;
     |     ^
   5 | }
     | ~
D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2>
```

Ici, nous avons corrigé l'erreur en ajoutant l'élément manquant.



```
Project Classes Debug hello.c error_example.c
1 #include <stdio.h>
2 int main() {
3     printf("This will cause an error");
4     return 0;
5 }
6
```

Après avoir recompilé, on remarque que l'avertissement a disparu.



```
D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2>gcc error_example.c -o error_example.exe

D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2>error_example.exe
This will cause an error

D:\Mega\Personal\Documents\The 5\Sim0\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice2>
```

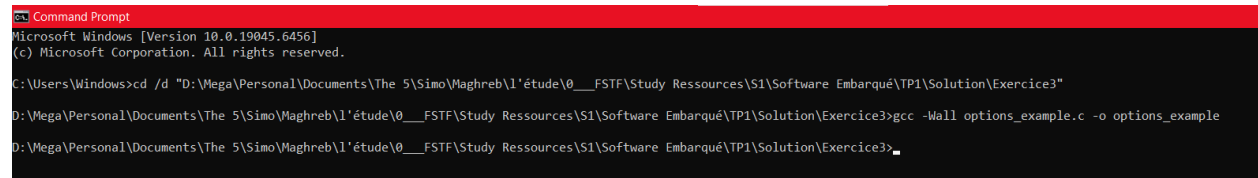
Exercice 3

L'objectif de cet exercice est de comprendre l'importance des options de compilation, en particulier l'option `-Wall`, qui permet d'afficher tous les avertissements possibles lors de la compilation d'un programme en C.

On commence par naviguer vers le répertoire suivant :

« **`cd /d "D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3`** »

Puis on compile le fichier avec la commande : « **`gcc -Wall options_example.c -o options_example`** »



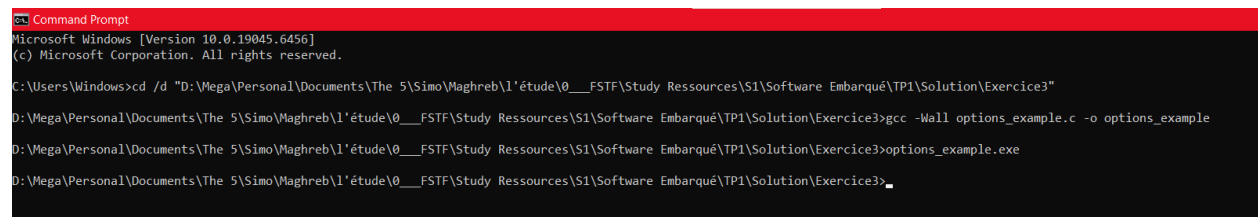
```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows>cd /d "D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3"

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3>gcc -Wall options_example.c -o options_example

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3>_
```

L'exécution de **`options_example.exe`** a été interrompue « `execution was aborted` ». Le comportement observé du programme est donc celui-ci.



```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows>cd /d "D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3"

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3>gcc -Wall options_example.c -o options_example

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3>options_example.exe

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice3>_
```

Exercice 4

Dans cet exercice, nous avons créé un projet contenant deux fichiers sources : **main.c** et **functions.c**.

Le fichier **Makefile** permet d'automatiser la compilation avec la commande **make**, ce qui facilite la gestion de plusieurs fichiers et évite de retaper manuellement les commandes GCC à chaque modification.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Windows>cd /d "D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice4"

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice4>make
gcc -Wall -c main.c -o main.o
gcc -Wall -c functions.c -o functions.o
gcc main.o functions.o -o program.exe

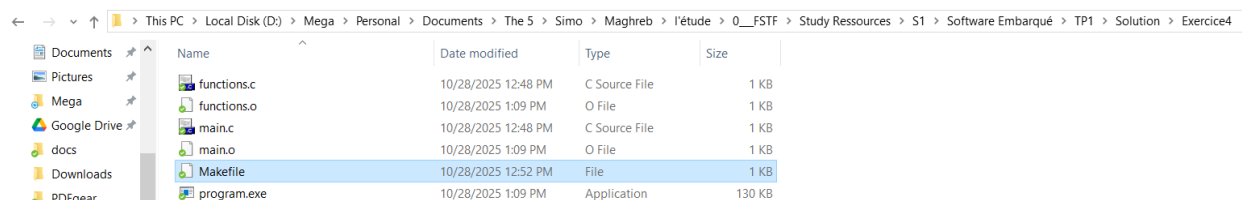
D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice4>program.exe
Hello from functions.c!

D:\Mega\Personal\Documents\The 5\Simo\Maghreb\l'étude\0__FSTF\Study Ressources\S1\Software Embarqué\TP1\Solution\Exercice4>
```

Voici le fichier **Makefile** :

```
Makefile
1  # Makefile for the project
2
3  # Compiler
4  CC = gcc
5
6  # Executable name
7  TARGET = program.exe
8
9  # Source files
10 SRC = main.c functions.c
11
12 # Object files (replace .c by .o)
13 OBJ = $(SRC:.c=.o)
14
15 # Compilation flags
16 CFLAGS = -Wall
17
18 # Default rule
19 all: $(TARGET)
20
21 # Link object files to create executable
22 $(TARGET): $(OBJ)
23     $(CC) $(OBJ) -o $(TARGET)
24
25 # Compile each .c file into .o
26 %.o: %.c
27     $(CC) $(CFLAGS) -c $< -o $@
28
29 # Clean generated files
30 clean:
31     del $(OBJ) $(TARGET)
```

Enfin, on peut voir les fichiers générés en suivant ce **Makefile**.



The screenshot shows a Windows File Explorer window with the address bar displaying the path: This PC > Local Disk (D:) > Mega > Personal > Documents > The 5 > Simo > Maghreb > l'étude > 0__FSTF > Study Ressources > S1 > Software Embarqué > TP1 > Solution > Exercice4. The file list contains the following items:

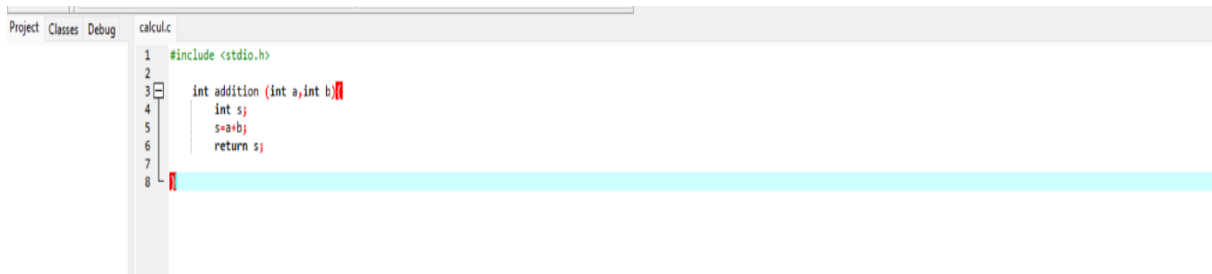
Name	Date modified	Type	Size
functions.c	10/28/2025 12:48 PM	C Source File	1 KB
functions.o	10/28/2025 1:09 PM	O File	1 KB
main.c	10/28/2025 12:48 PM	C Source File	1 KB
main.o	10/28/2025 1:09 PM	O File	1 KB
Makefile	10/28/2025 12:52 PM	File	1 KB
program.exe	10/28/2025 1:09 PM	Application	130 KB

Exercice 5

Dans cet exercice, nous allons créer deux fichiers : **calcul.c**, qui contient la fonction d'addition, et **main.c**, qui utilise cette fonction pour additionner deux nombres.

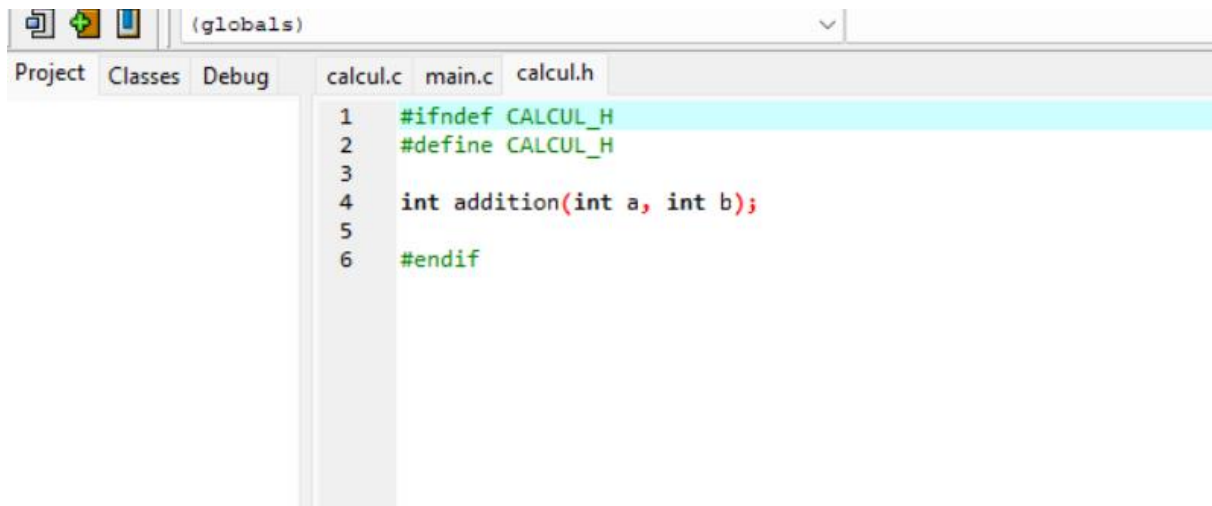
Un fichier **calcul.h** contient les déclarations des fonctions d'addition.

Voici le code de calcul.c :



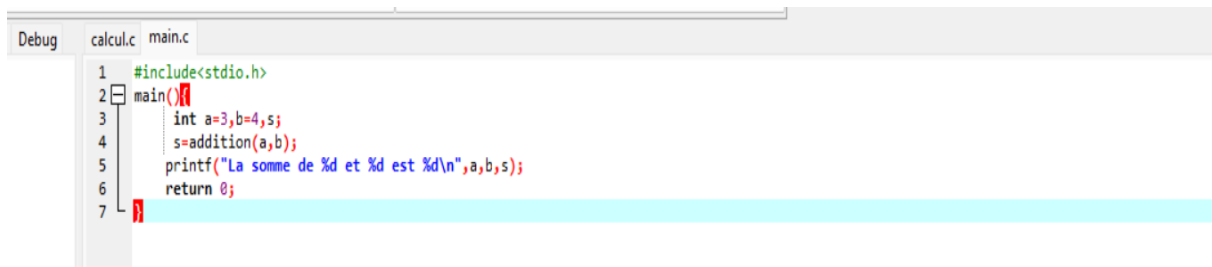
```
1 #include <stdio.h>
2
3 int addition (int a, int b)
4 {
5     int s;
6     s=a+b;
7     return s;
8 }
```

Voici le code de calcul.h :



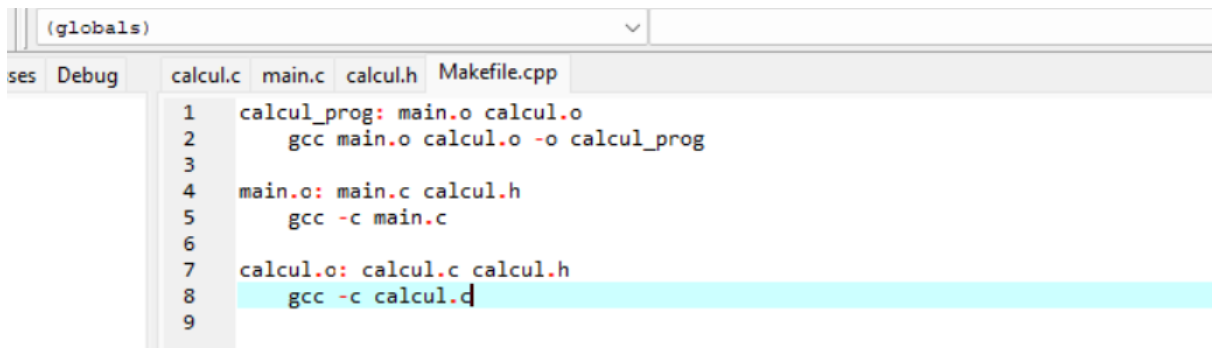
```
1 #ifndef CALCUL_H
2 #define CALCUL_H
3
4 int addition(int a, int b);
5
6 #endif
```

Voici le code de main.c :



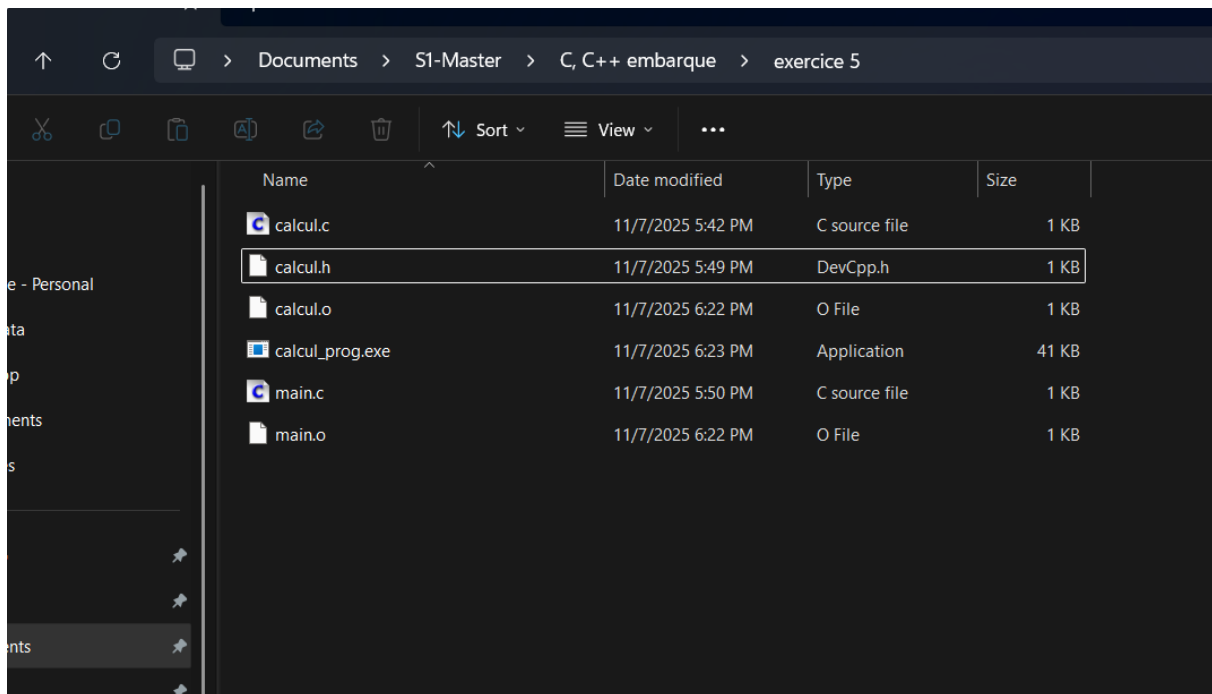
```
1 #include <stdio.h>
2 main()
3 {
4     int a=3,b=4,s;
5     s=addition(a,b);
6     printf("La somme de %d et %d est %d\n",a,b,s);
7     return 0;
8 }
```

Et voilà le code **Makefile** :



```
(globals)
calcul.c main.c calcul.h Makefile.cpp
1 calcul_prog: main.o calcul.o
2 gcc main.o calcul.o -o calcul_prog
3
4 main.o: main.c calcul.h
5 gcc -c main.c
6
7 calcul.o: calcul.c calcul.h
8 gcc -c calcul.c
9
```

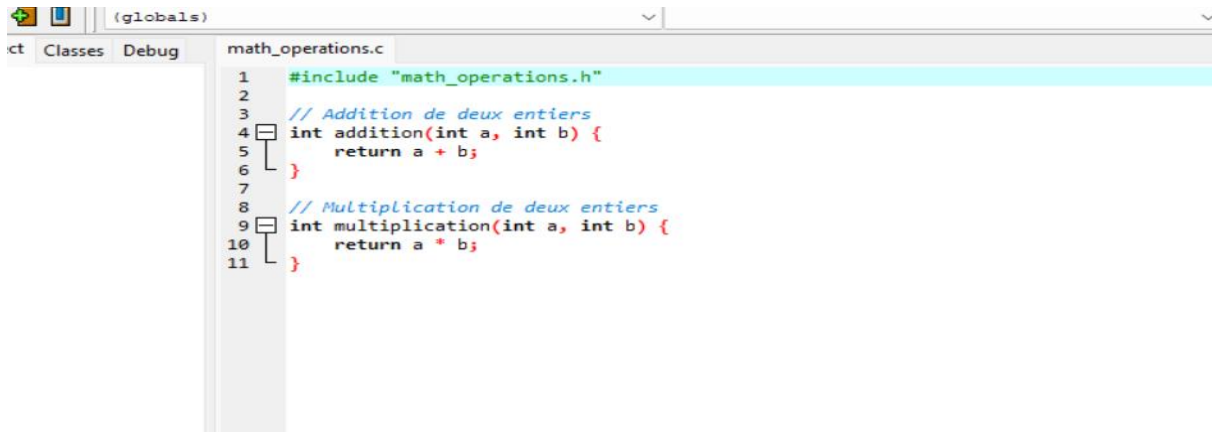
Après avoir compilé avec la commande : « gcc main.o calcul.o -o calcul_prog » nous obtenons les fichiers exécutables suivants :



Name	Date modified	Type	Size
calcul.c	11/7/2025 5:42 PM	C source file	1 KB
calcul.h	11/7/2025 5:49 PM	DevCpp.h	1 KB
calcul.o	11/7/2025 6:22 PM	O File	1 KB
calcul_prog.exe	11/7/2025 6:23 PM	Application	41 KB
main.c	11/7/2025 5:50 PM	C source file	1 KB
main.o	11/7/2025 6:22 PM	O File	1 KB

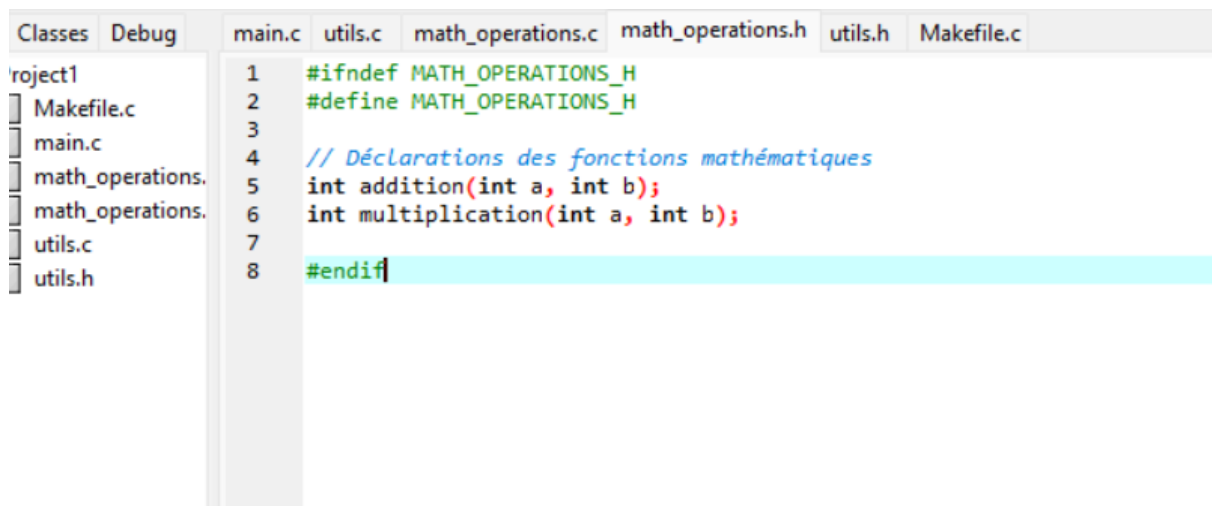
Exercice 6

Voici le code de **math_operations.c** :



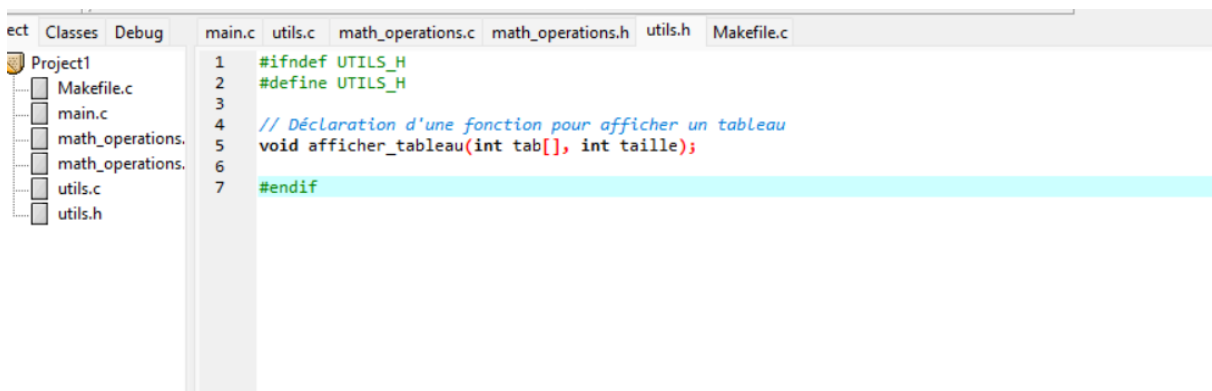
```
1  #include "math_operations.h"
2
3  // Addition de deux entiers
4  int addition(int a, int b) {
5      return a + b;
6  }
7
8  // Multiplication de deux entiers
9  int multiplication(int a, int b) {
10     return a * b;
11 }
```

Voici le code de **math_operations.h**



```
1  #ifndef MATH_OPERATIONS_H
2  #define MATH_OPERATIONS_H
3
4  // Déclarations des fonctions mathématiques
5  int addition(int a, int b);
6  int multiplication(int a, int b);
7
8  #endif
```

Voici le code de **Utils.h** :



```
1  #ifndef UTILS_H
2  #define UTILS_H
3
4  // Déclaration d'une fonction pour afficher un tableau
5  void afficher_tableau(int tab[], int taille);
6
7  #endif
```


Voici le code de **main.c**

```
[*] main.c  utils.c  math_operations.c  math_operations.h  utils.h  Makefile.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int a = 5, b = 3, taille = 5;
6      printf("Addition: %d + %d = %d\n", a, b, addition(a, b));
7      printf("Multiplication: %d * %d = %d\n", a, b, multiplication(a, b));
8      int tab1[] = {1, 2, 3, 4, 5};
9      afficher_tableau(tab1, taille);
10     return 0;
11 }
12
```

Nous avons constaté que compiler tous ces fichiers manuellement prendrait beaucoup de temps.

En utilisant un **Makefile**, on peut tout compiler instantanément avec une seule commande **make**.

```
(globals)
Project1  Classes  Debug  [*] main.c  utils.c  math_operations.c  math_operations.h  utils.h  Makefile.c
1  prog: main.o utils.o math_operations.o
2      gcc main.o utils.o math_operations.o -o prog
3
4  main.o: main.c utils.h math_operations.h
5      gcc -c main.c
6
7  utils.o: utils.c utils.h
8      gcc -c utils.c
9
10 math_operations.o: math_operations.c math_operations.h
11      gcc -c math_operations.c
12
```

Nous obtenons les fichiers exécutables suivants :

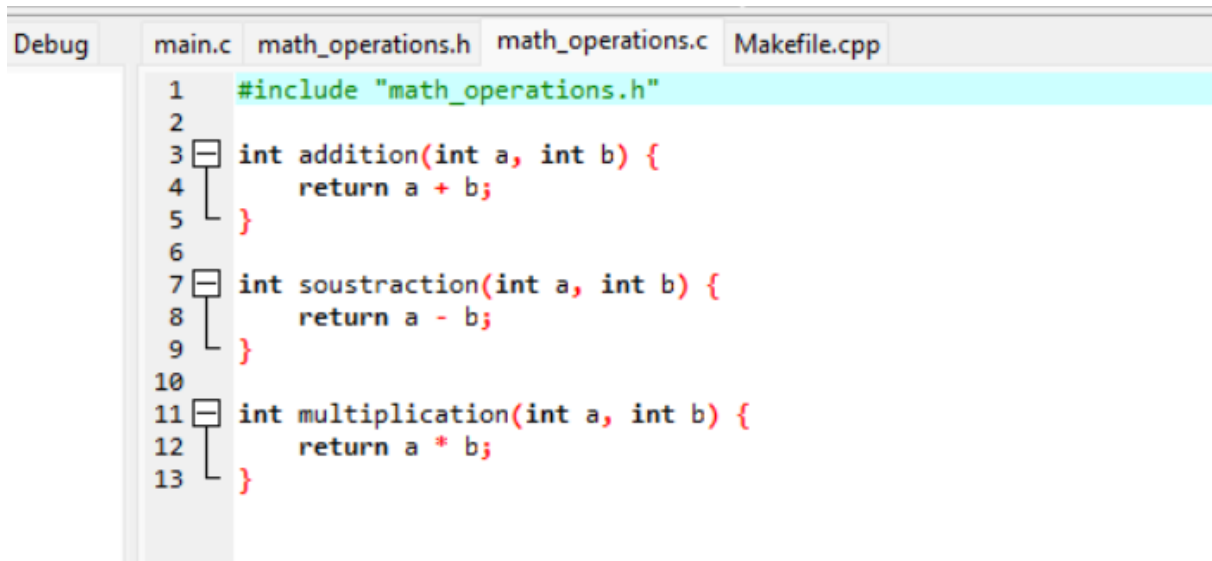
Name	Date modified	Type	Size
main.c	11/7/2025 7:07 PM	C source file	1 KB
main.o	11/7/2025 7:12 PM	O File	2 KB
Makefile.c	11/10/2025 9:36 PM	C source file	1 KB
Makefile.win	11/7/2025 7:16 PM	WIN File	2 KB
math_operations.c	11/7/2025 7:00 PM	C source file	1 KB
math_operations.h	11/7/2025 7:07 PM	DevCpp.h	1 KB
math_operations.o	11/7/2025 7:11 PM	O File	1 KB
prog.exe	11/7/2025 7:13 PM	Application	42 KB
Project1.dev	11/7/2025 7:23 PM	Dev-C++ Project F...	2 KB
Project1.layout	11/7/2025 7:23 PM	LAYOUT File	1 KB
utils.c	11/7/2025 7:00 PM	C source file	1 KB
utils.h	11/7/2025 6:59 PM	DevCpp.h	1 KB
utils.o	11/7/2025 7:11 PM	O File	1 KB

Exercice 7

Dans cet exercice, nous avons créé une **bibliothèque statique** à partir du fichier **math_operations.c**.

Cette bibliothèque, appelée **libmath.a**, est ensuite liée avec **main.c**.

Voici le code de **math_operations.c** :



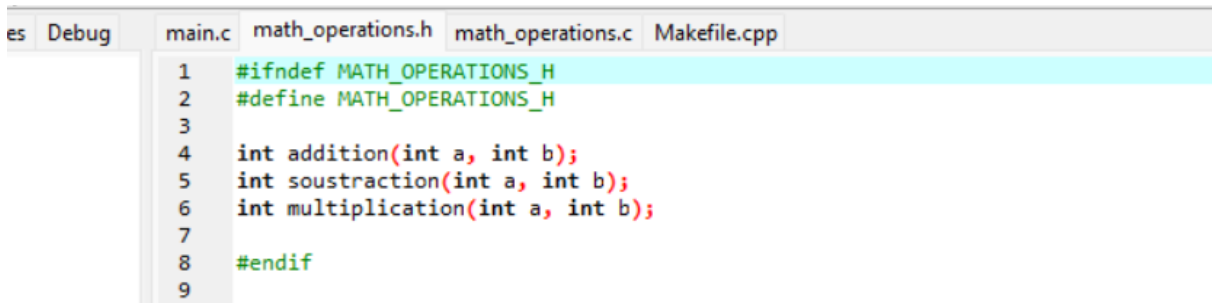
```
1  #include "math_operations.h"
2
3  int addition(int a, int b) {
4      return a + b;
5  }
6
7  int soustraction(int a, int b) {
8      return a - b;
9  }
10
11 int multiplication(int a, int b) {
12     return a * b;
13 }
```

Voici le code de **main.c** :



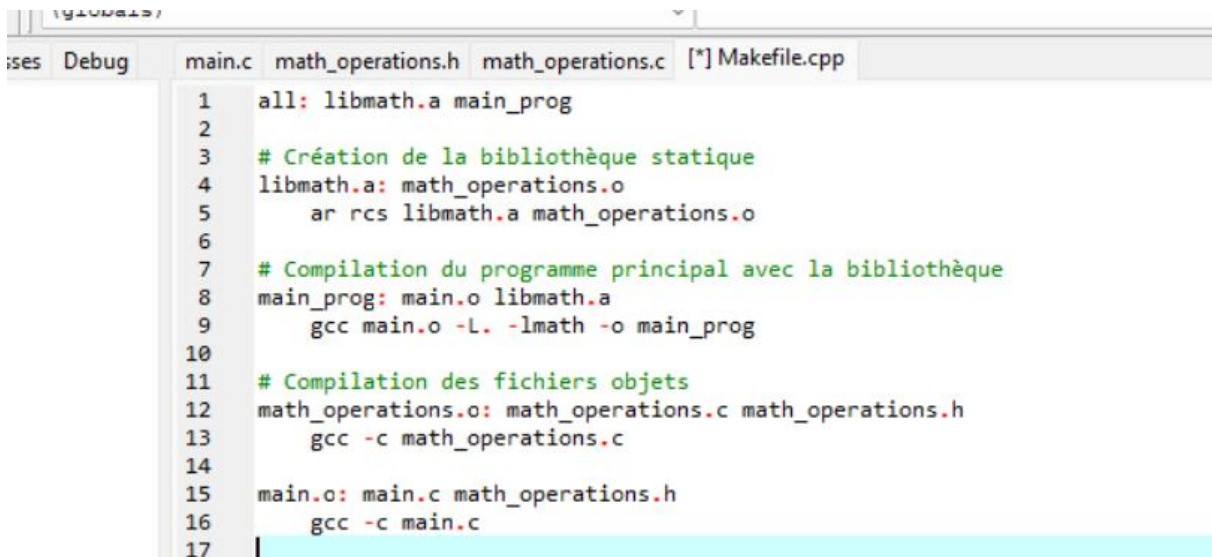
```
1  #include <stdio.h>
2  #include "math_operations.h"
3
4  int main() {
5      int a = 8, b = 3;
6
7      printf("Addition: %d + %d = %d\n", a, b, addition(a, b));
8      printf("Soustraction: %d - %d = %d\n", a, b, soustraction(a, b));
9      printf("Multiplication: %d * %d = %d\n", a, b, multiplication(a, b));
10
11     return 0;
12 }
13
```

Voici le code de **math_operations.h** :



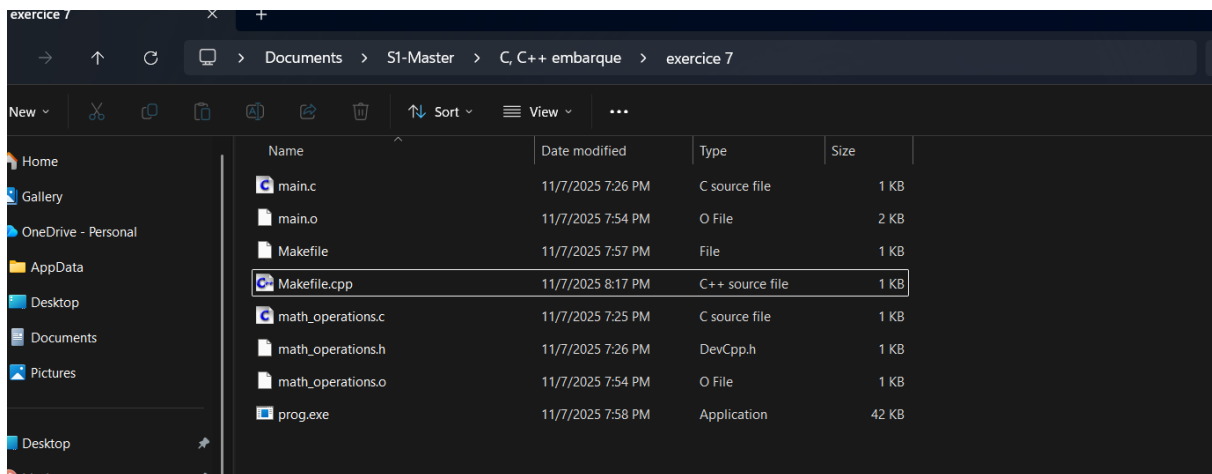
```
1 #ifndef MATH_OPERATIONS_H
2 #define MATH_OPERATIONS_H
3
4 int addition(int a, int b);
5 int soustraction(int a, int b);
6 int multiplication(int a, int b);
7
8 #endif
9
```

En utilisant un **Makefile**, on peut tout compiler instantanément avec une seule commande **make** :



```
1 all: libmath.a main_prog
2
3 # Création de la bibliothèque statique
4 libmath.a: math_operations.o
5     ar rcs libmath.a math_operations.o
6
7 # Compilation du programme principal avec la bibliothèque
8 main_prog: main.o libmath.a
9     gcc main.o -L. -lmath -o main_prog
10
11 # Compilation des fichiers objets
12 math_operations.o: math_operations.c math_operations.h
13     gcc -c math_operations.c
14
15 main.o: main.c math_operations.h
16     gcc -c main.c
17
```

Nous obtenons les fichiers exécutables suivants :



Name	Date modified	Type	Size
main.c	11/7/2025 7:26 PM	C source file	1 KB
main.o	11/7/2025 7:54 PM	O File	2 KB
Makefile	11/7/2025 7:57 PM	File	1 KB
Makefile.cpp	11/7/2025 8:17 PM	C++ source file	1 KB
math_operations.c	11/7/2025 7:25 PM	C source file	1 KB
math_operations.h	11/7/2025 7:26 PM	DevCpp.h	1 KB
math_operations.o	11/7/2025 7:54 PM	O File	1 KB
prog.exe	11/7/2025 7:58 PM	Application	42 KB

Conclusion

La réalisation de cette série d'exercices nous a permis de maîtriser les concepts fondamentaux de la compilation en C.

Nous avons compris le rôle du compilateur GCC, la détection des erreurs, l'utilisation des options de compilation et la création d'un Makefile pour automatiser les tâches répétitives. Cette approche structurée facilite le développement, la maintenance et la portabilité des programmes en environnement embarqué.

Ainsi, cette série constitue une étape essentielle vers la programmation efficace et professionnelle en langage C.